

4

EL NIVEL DE MICROPROGRAMACION

La frontera entre el ~~hardware~~ y el ~~software~~ no está bien definida y, además, varía constantemente. Las primeras computadoras tenían instrucciones para hacer operaciones aritméticas y booleanas, corrimientos, comparaciones, iteraciones y otras que eran realizadas directamente por el hardware. Para cada instrucción había un circuito específico que la ejecutaba. En teoría al menos, se podía desatornillar el panel trasero e identificar los componentes electrónicos que realizaban la instrucción de dividir, por ejemplo.

En una moderna computadora multinivel ya no es posible aislar los circuitos de la división porque no existen. Todas las instrucciones en el nivel de máquina convencional (por ejemplo: aritméticas, booleanas, de miento, de comparación y de bucle) son realizadas paso a paso por un intérprete que se ejecuta en el nivel de microprogramación. El equivalente moderno de buscar los circuitos de la división es obtener un listado del microprograma y examinar la porción que interpreta la instrucción de dividir.

Aunque los programas de cualquier nivel pueden ser ejecutados por un programa intérprete que, a su vez, puede ser llevado a cabo por otro intérprete, esta jeraquía no puede continuar indefinidamente. En el nivel inferior deberá haber una máquina física, con circuitos integrados, fuentes de alimentación y otros objetos del hardware. Estos elementos fueron el tema del capítulo precedente. En este capítulo estudiaremos cómo el microprograma controla los componentes del hardware e interpreta el nivel de máquina ordinaria (convencional). En el capítulo 8 se estudiará una clase de máquinas no microprogramadas (las máquinas RISC).

Como la arquitectura del nivel de microprogramación, denominada **microarquitectura**, está definida por el hardware, suele ser primitiva y difícil de programar. Por ejemplo, a menudo son importantes las consideraciones de tiempos. Esto llevó a Rosin (1974) a definir la microprogramación como "el diseño de sistemas más o menos razonables por medio de la interpretación sobre máquinas irracionales".

El nivel de microprogramación tiene una función específica: ejecutar intérpretes de otras máquinas virtuales (más o menos razonables). El objetivo natural de diseño es la obtención de una organización considerablemente optimizada para la extracción, examen y ejecución de instrucciones del nivel de máquina convencional y, en algunos casos, de otras más complejas. En este capítulo examinaremos los principios y soluciones intermedias que hay que considerar en el diseño y organización de este nivel.

Empezaremos nuestro estudio del nivel de microprogramación revisando brevemente los elementos básicos estudiados en el capítulo 3, ya que son parte de la arquitectura del nivel de microprogramación y, por tanto, de interés para el microprogramador (persona que escribe microprogramas, no un programador pequeño). Luego entraremos en el meollo del tema, explicando de modo pormenorizado cómo pueden construirse instrucciones complejas a partir de secuencias de instrucciones más primitivas. La explicación se apoyará en un ejemplo desarrollado detalladamente. Después examinaremos los factores que deben tenerse en cuenta al diseñar el nivel de microprogramación de una computadora, para entender mejor por qué tiene determinada estructura. También se abordarán algunas formas para mejorar el desempeño de la computadora. Por último, se examinará el nivel de microprogramación de los dos ejemplos que se han venido usando, las familias Intel y Motorola.

4.1. REPASO SOBRE EL NIVEL DE LOGICA DIGITAL

El trabajo del microprogramador es escribir programas que controlen los registros, buses, unidades aritméticas y lógicas, memorias y otros componentes del hardware de las máquinas. En el capítulo anterior estudiamos estos dispositivos; ahora simplemente vamos a revisarlos para tenerlos presentes. Después del repaso diremos algunas cosas sobre las diferentes formas conceptuales de encapsular los componentes.

4.1.1. Registros

Un registro es un dispositivo capaz de almacenar información. El nivel de microprogramación siempre dispone de registros para guardar la información que se necesita en el procesamiento de la instrucción en curso de interpretación.

Número de bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	0	0	0	0	1	1	0	0	1	0	1	1	0	0	1	0

Fig. 4-1. Un registro de 16 bits puede guardar 16 bits de información.

Desde el punto de vista conceptual, los registros son lo mismo que la memoria principal; la diferencia estriba en que los registros están ubicados en el procesador mismo y, por tanto, se puede acceder a ellos en lectura y escritura más rápidamente que a la memoria principal, la cual suele estar fuera del chip (pastilla). Normalmente, las máquinas mayores y más caras tienen un número mayor de registros que las más pequeñas y económicas; éstas tienen que usar la memoria principal para guardar los resultados intermedios. En algunas computadoras el nivel de microprogramación dispone de un grupo de registros numerados desde 0, 1, 2..., hasta $n-1$, denominado **memoria local** o **memoria de anotaciones**.

Un registro puede caracterizarse por un único número, es decir, por cuántos bits puede guardar. La figura 4-1 muestra un registro de 16 bits con la convención de numeración de bits usada en este libro. La información que se guarda en un registro permanece en él hasta que otra la reemplaza. El proceso de leer la información de un registro no afecta a su contenido. En otras palabras, cuando se lee un registro, lo que se hace es una copia de su contenido, dejando inalterado el original.

4.1.2. Buses

Un bus es un conjunto de alambres que se usan para transmitir señales en paralelo. Por ejemplo, los buses se utilizan para permitir que el contenido de un registro se copie en otro. A diferencia de los buses del sistema que se estudiaron en el capítulo 3, éstos sólo conectan dos dispositivos, de modo que no hay necesidad de líneas de direcciones o líneas extensivas de control. Por lo general son suficientes las n líneas de datos y una o dos de control. Se emplean buses, pues la transmisión paralela de todos los bits a la vez, es mucho más rápida que la transmisión serial bit por bit.

Un bus puede ser unidireccional o bidireccional. Un bus unidireccional puede transferir información en un solo sentido; en cambio uno bidireccional puede transferirla en los dos sentidos, pero no simultáneamente. Los unidireccionales suelen usarse para conectar dos registros, uno de los cuales siempre es fuente y el otro siempre destino. Los bidireccionales suelen usarse cuando hay una colección de registros, cualquiera de los cuales puede ser fuente o destino.

Muchos dispositivos tienen la posibilidad de conectarse y desconectarse eléctricamente de los buses a los que están conectados físicamente. Estas conexiones pueden abrirse y cerrarse en cuestión de nanosegundos. Un bus cuyos

dispositivos tengan esta propiedad se llama **bus triestado**, porque cada línea puede estar a 0, a 1 o desconectada. Suele utilizarse cuando hay muchos dispositivos que pueden suministrar información a un bus.

En la mayoría de las microarquitecturas algunos registros están conectados a uno o más buses de entrada y a uno o más de salida. La figura 4-2(a) muestra

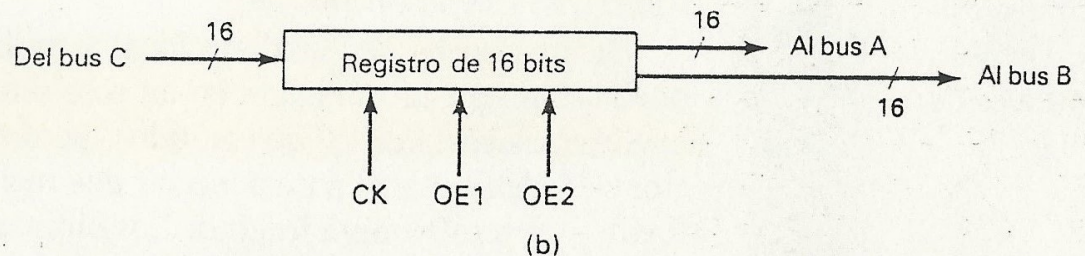
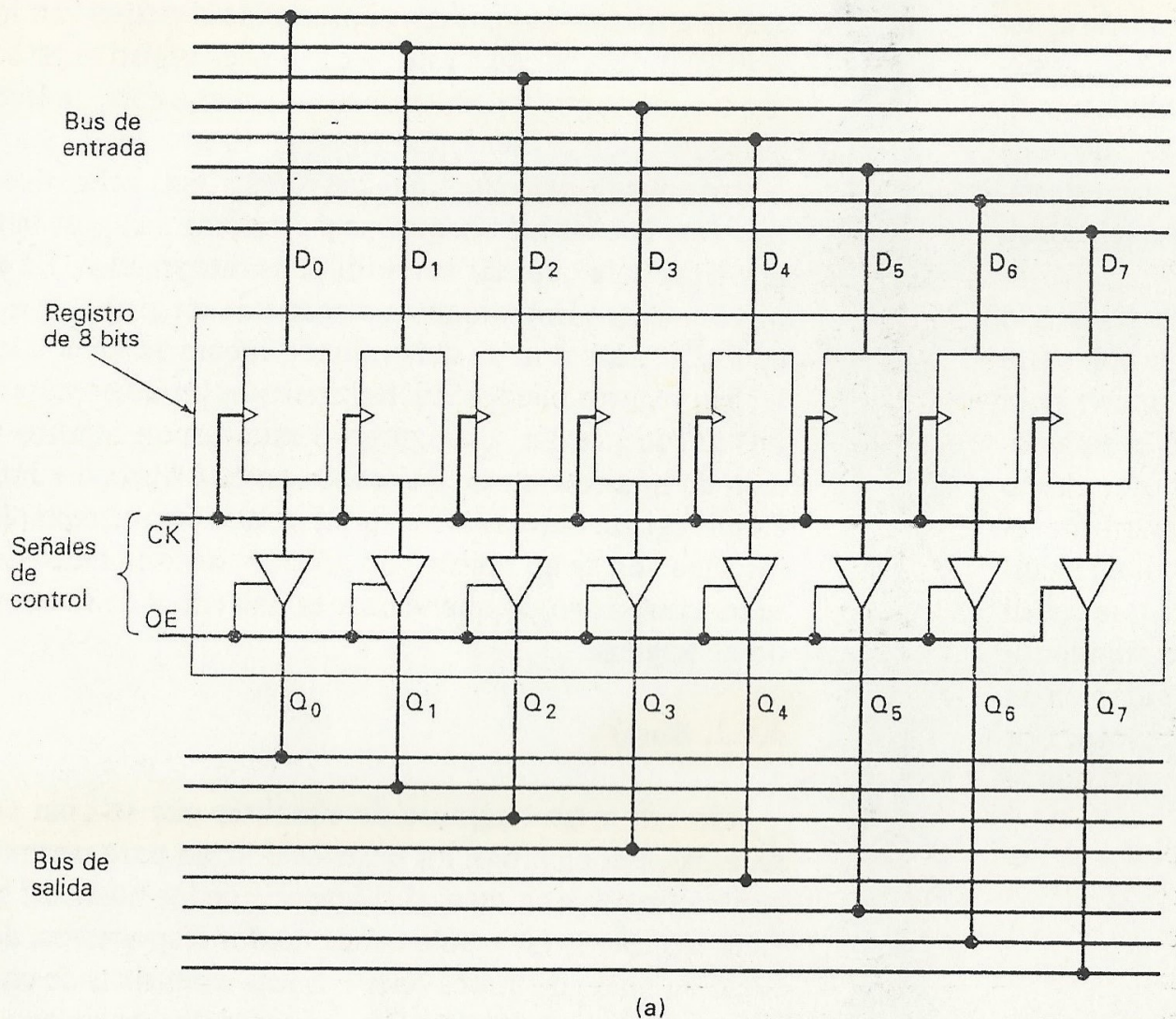


Fig. 4-2. (a) Detalle de un registro de 8 bits conectado a bus (conductor) de entrada y a otro de salida. (b) Representación simbólica de un registro de 16 bits con un bus de entrada y dos de salida.

un esquema de un registro de 8 bits conectado a un bus de entrada y a otro de salida. El registro consta de ocho flip-flops del tipo D conectados, cada uno de ellos, al bus de salida por medio de un buffer no inversor. Cada uno contiene 1 bit. El registro tiene dos señales de control CK (reloj, que realmente quiere decir "carga registro") y DE (permiso de SALida), conectadas ambas a todos los flip-flops. Normalmente ambas señales se hallan en su estado de reposo, pero a veces pueden ser **activadas**, causando alguna acción.

Cuando CK están desactivada, el contenido del registro no es afectado por las señales del bus. En cambio, cuando se activa, el registro se carga con el contenido del bus de entrada. Cuando DE está desactivada, el registro está desconectado del bus de salida y desaparece respecto a los restantes registros conectados. Cuando DE está activo, el contenido de registro pasa al bus de salida.

Si otro registro, *R*, tuviera su entrada conectada al bus de salida de nuestro registro, podría transferirse información de éste a *R*. Para hacerlo, hay que activar DE y mantenerlo activo tanto tiempo como sea necesario para que la salida al bus se estabilice. Después se debe activar la línea CK de *R*, con lo que éste se carga con lo que hay en el bus. En el nivel de microprogramación son frecuentes estas operaciones de conexión de un registro a un bus para que otro lo lea, como veremos en seguida. He aquí un segundo ejemplo de registros y buses: la figura 4-2(b) muestra un registro de 16 bits con dos buses de salida, cada uno de ellos controlado por una señal DE diferente.

4.1.3. Multiplexores y decodificadores

Los circuitos que tienen una o más líneas de entrada y que calculan uno o varios valores de salida determinados únicamente por las entradas actuales se llaman **circuitos combinacionales**. Dos de los más importantes son los multiplexores y los decodificadores. Un **multiplexor** tiene 2^n entradas de datos (líneas individuales o buses), una salida de datos de la misma anchura que la entrada y una entrada de control de n bits que selecciona una de las entradas y la encamina a la salida. La figura 4-3(a) muestra un multiplexor con dos buses de entrada. La señal de control de un bit selecciona *A* o *B* como salida. La figura 3-12 muestra el circuito de un multiplexor de 8 entradas.

El inverso del multiplexor es el **demultiplexor**, que encamina su única entrada a una de sus 2^n salidas, según el valor que tengan sus n líneas de control.

Otro circuito combinacional importante es el **decodificador**, que tiene n líneas de entrada y 2^n de salida, numeradas desde 0 hasta $2^n - 1$. Si el número que hay en las líneas de entrada es k , entonces se pone a 1 la salida k , permaneciendo las demás a 0. Un decodificador tiene siempre una y una sola salida a 1, estando a 0 las restantes. La figura 4-3(b) ilustra simbólicamente un decodificador de 4 a 16. La figura 3-14 muestra el circuito de un decodificador de 3 a 8.

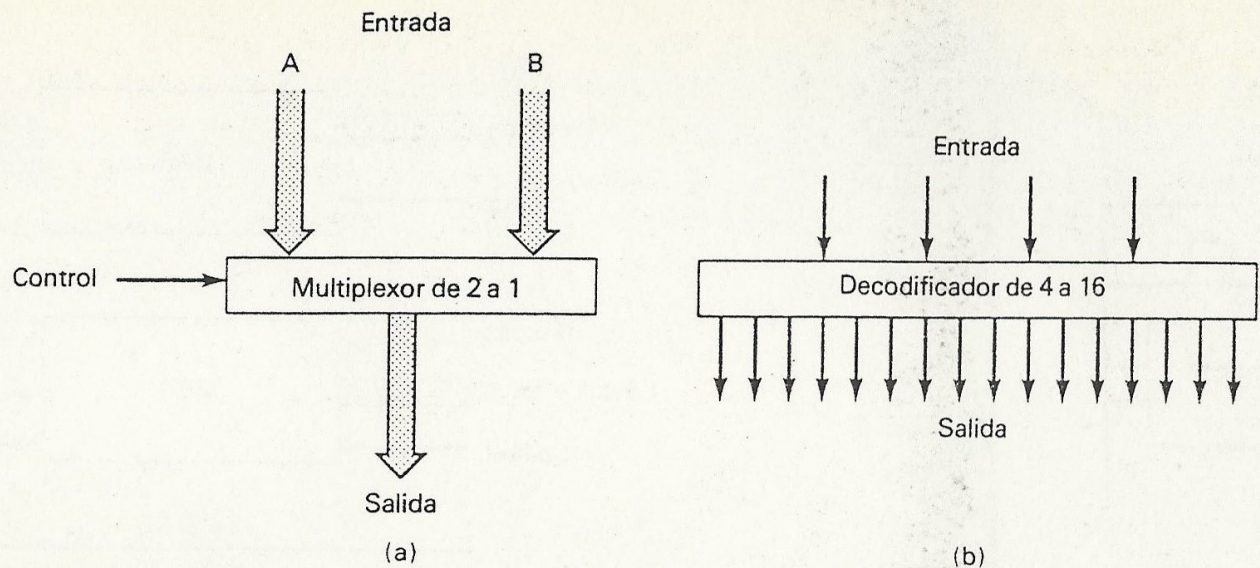


Fig. 4-3. (a) Multiplexor de 2 a 1. (b) Decodificador de 4 a 16.

El inverso del decodificador es el **codificador**, que tiene 2^n entradas y n salidas. Sólo puede haber una línea de entrada a 1 y su número, en binario, aparece en la salida.

4.1.4. Unidades aritméticas y lógicas y registros de corrimiento

Toda computadora necesita algún medio para hacer operaciones aritméticas. El circuito más simple es un sumador, que toma dos entradas de n bits y produce su suma como salida. Un circuito aritmético más general es la **ALU (Unidad Aritmética y Lógica)**. También tiene dos entradas y una salida de datos, pero además tiene algunas entradas y salidas de control. La de la figura 4-4(a) tiene dos bits de función, F_0 y F_1 , que determinan qué función se ha de realizar. La ALU que vamos a usar en nuestro ejemplo puede calcular $A + B$, $\text{AND } B$, A y $\bar{A}$. Las dos primeras no requieren explicación; la tercera simplemente copia A en la salida; la cuarta obtiene A negando (todos sus bits invertidos). La tercera función puede parecer inútil, pero veremos su utilidad después. La figura 3-20 muestra una ALU de cuatro funciones que opera sobre datos de un bit. Replicando este circuito n veces y conectando el acarreo del bit i al del $i + 1$, obtendremos una ALU de n bits que realice las operaciones AND, OR, NOT y suma.

Una ALU también puede tener salidas de control. Salidas típicas son aquellas que se ponen a 1 cuando la salida de la ALU es negativa o cero, cuando hay acarreo del bit más significativo o cuando ha ocurrido un desbordamiento. El ejemplo de la figura 4-4(a) tiene dos salidas de control: N , para indicar que la salida de la ALU es negativa y Z , que indica que es cero. El bit N es simplemente una copia del más significativo de la salida, mientras que el bit Z es el NOR de todos los bits de salida.

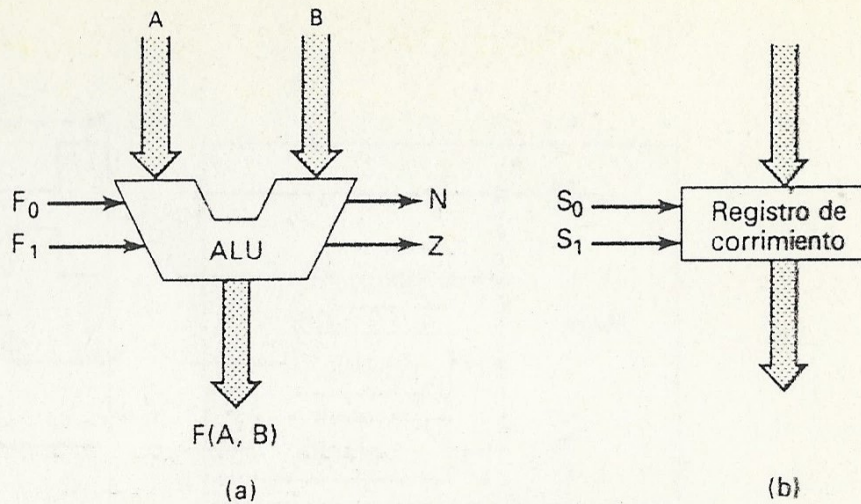


Fig. 4-4. (a) ALU. (b) Registro de corrimiento.

Aunque algunas ALU pueden realizar también operaciones de desplazamiento, la mayoría de las veces es necesaria una unidad específica. Este circuito puede desplazar su entrada un bit a la izquierda o a la derecha, o también no realizar ningún registro de corrimiento. La figura 4-4(b) muestra el símbolo que utilizaremos para registros de corrimiento; la figura 3-17 muestra el circuito de un desplazador algo más sencillo.

4.1.5. Relojes

Los circuitos de las computadoras funcionan normalmente al ritmo de un reloj, dispositivo que emite una secuencia periódica de impulsos. Estos impulsos definen los ciclos de máquina. Durante cada ciclo tiene lugar alguna actividad básica, como la ejecución de una microinstrucción. A menudo es útil dividir los ciclos en subciclos, para que las partes de una microinstrucción puedan realizarse en un orden bien definido. Por ejemplo, las entradas en la ALU deben estar disponibles y estabilizadas antes de que pueda almacenarse su salida.

La figura 4-5(a) simboliza un reloj de cuatro salidas. La de arriba es la salida principal; las otras tres se derivan de ella al someterla a distintos retardos. La salida principal [línea superior de la figura 4-5(b)] tiene un ancho de un cuarto de ciclo. Las otras tres están retrasadas uno, dos y tres veces el ancho del impulso. El resultado es un circuito que divide cada ciclo en cuatro subciclos idénticos. Para más detalles, véase la figura 3-21.

El diseñador de computadoras puede hacer que cuatro transiciones diferentes tengan lugar en cierto orden dentro de un ciclo activando cada una de ellas con el AND lógico de la señal habilitadora y una línea de reloj distinta. Primero se disparará la transición asociada a la línea principal de reloj, luego la que esté asociada a la línea de reloj de menor retardo y así sucesivamente.

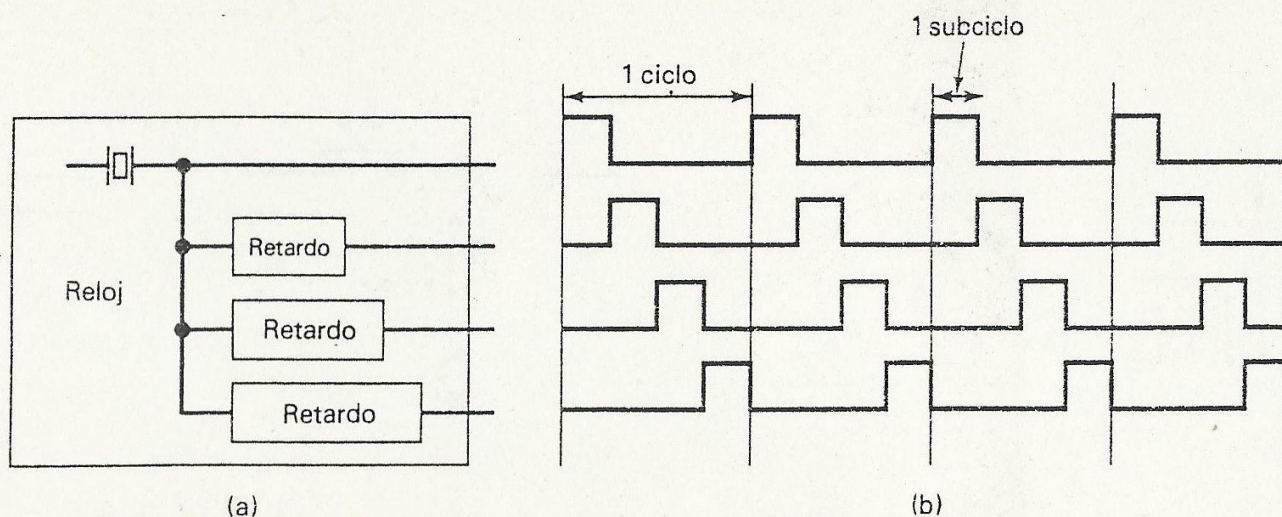


Fig. 4-5. (a) Reloj con cuatro salidas. (b) Su cronograma.

4.1.6. Memoria principal

Los procesadores deben ser capaces de leer y escribir en memoria. La mayoría de las computadoras tienen un bus (conductor) de direcciones, otro de datos y otro de control para la comunicación entre la CPU y la memoria. Para leer de la memoria, la CPU pone una dirección en el bus de direcciones y ajusta adecuadamente las señales de control; por ejemplo, activando la señal RD (lectura). Luego, la memoria pone el dato pedido en el bus de datos. En algunas computadoras la lectura y la escritura en memoria son sincronicas; es decir, la memoria debe responder dentro de un plazo fijo. En otras puede tomarse el tiempo que quiera, señalizando por medio de una línea de control la presencia de datos cuando termine.

Las escrituras en memoria se hacen en forma parecida. La CPU pone en el bus de datos el dato a escribir y en el bus de direcciones la dirección donde se quiere guardar, activando después WR (escritura). Una alternativa de tener las señales RD y WR sería tener una señal MREQ, de petición a la memoria y otra RW, que distinga entre la lectura y la escritura.

Un acceso a memoria es siempre considerablemente más largo que el que se necesita para ejecutar una sola microinstrucción. En consecuencia, el microprograma debe mantener los valores correctos en los buses de datos y direcciones durante varias microinstrucciones. Para simplificar esta tarea, a menudo es conveniente tener dos registros, el **MAR** (Registro de Dirección de Memoria) y el **MBR** (Registro de Intercambio de Memoria), a los que se conectan los buses de direcciones y de datos, respectivamente. En este libro conviene disponer los buses como se indica en la figura 4-6. Ambos registros se colocan entre la CPU y el bus del sistema. El bus de direcciones es unidireccional en ambos lados y se carga desde el lado de la CPU, activando la línea de control. Las líneas del bus